

Machine Learning (CE 40717)

Fall 2024

Ali Sharifi-Zarchi

CE Department
Sharif University of Technology

December 3, 2024

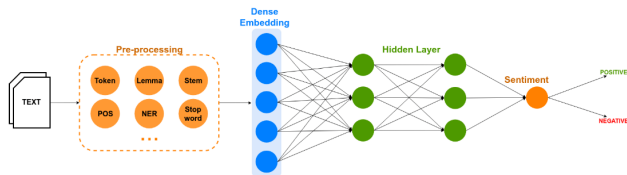


① Introduction

② Word2Vec

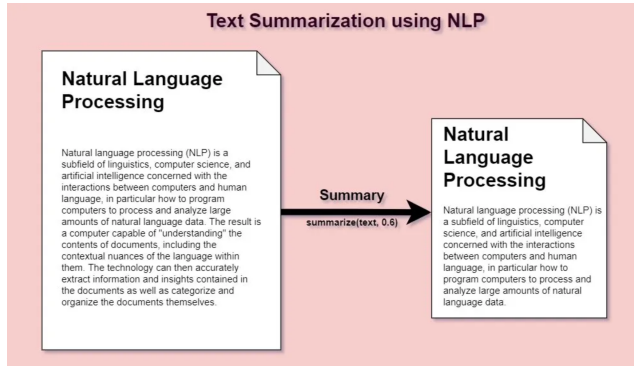
③ References

- Language is central to human interaction; many of our daily activities revolve around text and language.
- Natural Language Processing (NLP) enables computers to understand and generate human language.



- Determines the sentiment (e.g., positive or negative) expressed in a text.

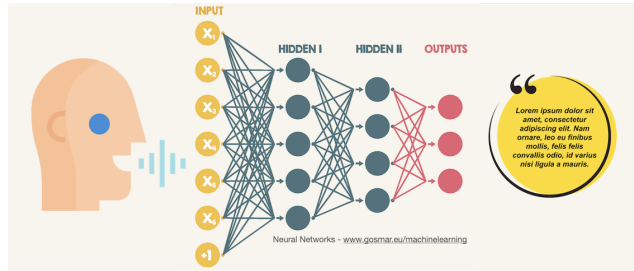
Figure adapted from www.mdpi.com/2079-9292/9/3/483



- Automatically generates a concise summary of longer text.

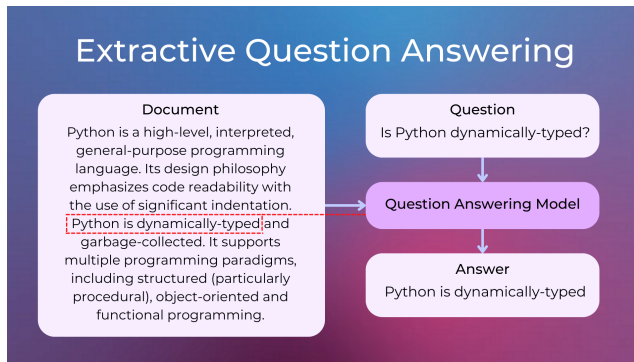
Figure adapted from turbolab.in/types-of-text-summarization-extractive-and-abstractive-summarization-basics/

Speech Recognition



- Converts spoken language into text.

Question Answering



- Answers questions based on a given text or dataset.

- To process text effectively, the first step is to represent words in a way that models can understand.
- We need to transform words into vectors or dense representations to capture their meaning and relationships.
- This is crucial for enabling machines to understand and use language as humans do.

Motivation and One-Hot Encoding

- Traditional models like one-hot encoding lack semantic understanding and fail to capture word relationships.
- One-hot encoding represents words as binary vectors where only one position is 1, making the representation sparse and non-semantic.
- We need dense, semantic word representations to improve natural language processing tasks.

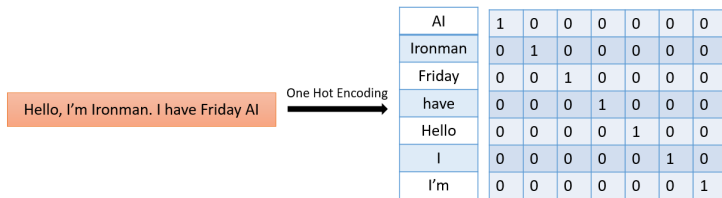


Figure adapted from medium.com/@kalyan45/natural-language-processing-one-hot-encoding-5b31f76b09a0

100

- For example, given a vocabulary of 5 words:
 - apple = [1, 0, 0, 0, 0]
 - banana = [0, 1, 0, 0, 0]
 - cherry = [0, 0, 1, 0, 0]
 - date = [0, 0, 0, 1, 0]
 - elderberry = [0, 0, 0, 0, 1]
- The length of the one-hot vector depends on the number of unique words in the vocabulary.

Strengths and Limitations of One-Hot Encoding

- **Strengths:**

- One-hot encoding is a simple and intuitive representation that can be effective in certain models, especially smaller neural networks.
- It requires minimal computation and works well for small vocabularies or categorical features in simpler tasks.

- **Limitations:**

- One-hot encoding does not capture semantic relationships, so similar words (e.g., `hotel` and `motel`) appear unrelated.
- The vectors are sparse, containing mostly zeros, making it inefficient for large vocabularies and prone to overfitting due to the large number of parameters in downstream models.

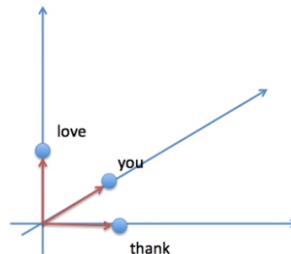
- **Cosine Similarity:**

1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 26

Example: One-Hot Encoding for Different Words

	thank	you	love
thank	1	0	0
you	0	1	0
love	0	0	1

unique word	encoding
thank	[1, 0, 0]
you	[0, 1, 0]
love	[0, 0, 1]

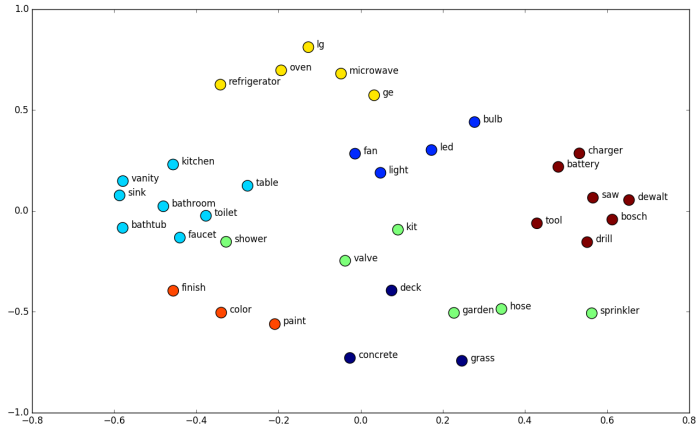


- Different words can be perpendicular to each other in space regardless of their meanings.

Figure adapted from andreaperlato.com/theorypost/introduction-to-word2vec/

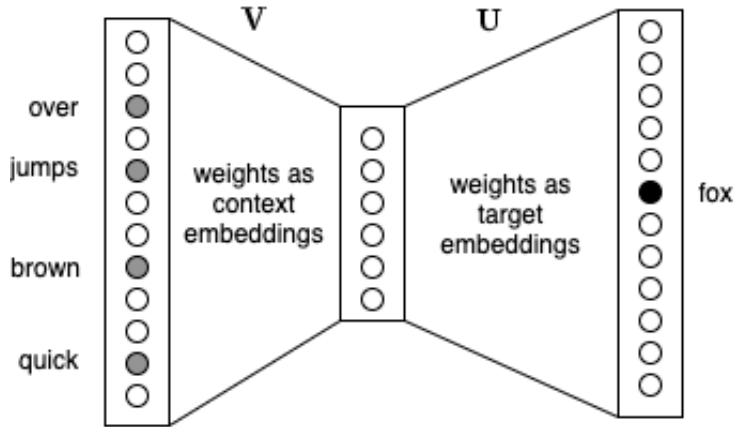
- To process text data, we need to represent words in a form that a machine can understand—numerical vectors.
- Word2Vec uses a neural network to learn **word embeddings** that capture semantic similarities.
- These embeddings allow words with similar meanings to be represented by vectors close to each other in a high-dimensional space.

Example: Word Embedding



- Word2Vec is a shallow neural network, with an input, hidden, and output layer.
- It takes in a target word and learns to predict either the surrounding context words or the target word from a set of context words.
- Through training, the network adjusts weights to create meaningful vector representations of words.

Word2Vec as a Neural Network



Expected Outcome of Word2Vec

- Word2Vec aims to create a vector space where words with similar meanings or contexts are located close to each other.
- **Expected Result:** Semantically related words—such as “king” and “queen” or “dog” and “puppy”—should have similar vector representations.
- This proximity allows for various NLP tasks, such as:
 - **Synonym detection:** Identifying words with similar meanings.
 - **Analogy tasks:** Solving analogies by vector arithmetic (e.g., “king” - “man” + “woman” “queen”).
 - **Clustering of concepts:** Grouping related concepts together in the embedding space.
- By representing words in this way, Word2Vec enables models to make use of semantic relationships between words.

Expected Outcome of Word2Vec



Hypothetical features to understand word embeddings

Word2Vec: Contextual Word Representation

- The core idea is based on distributional semantics: "You shall know a word by the company it keeps."
- Word2Vec uses two main algorithms for learning word vectors:
 - Continuous Bag of Words (CBOW)
 - Skip-gram Model

① Introduction

② Word2Vec

Continuous Bag of Words (CBOW)

Skip-gram Model

Word Embedding Visualization

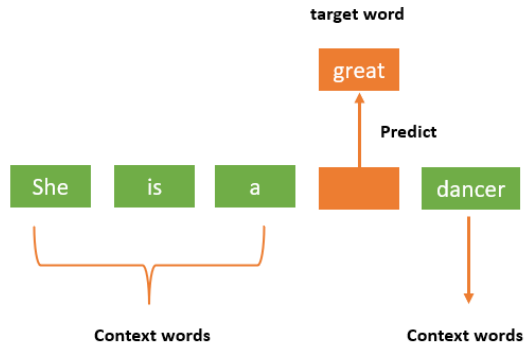
Word Analogy

③ References

CBOW: How It Works

- CBOW predicts the target word using the context (surrounding words) in a fixed window.
- For each word in the corpus, CBOW takes a set of context words and predicts the center word.
- Example: Given the context words {"the", "brown", "fox", "over"}, CBOW predicts the center word "jumps."
- CBOW tends to perform better on smaller datasets and is computationally more efficient.

Example: Continuous Bag of Words



① Introduction

② Word2Vec

Continuous Bag of Words (CBOW)

Skip-gram Model

Word Embedding Visualization

Word Analogy

③ References

Skip-gram: How It Works

- Skip-gram is the reverse of CBOW. It predicts the surrounding context words given a target word.
- For each word w_t , the model predicts the words in the window of size m around it (e.g., words $w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}$).
- Example: If the center word is “jumps,” Skip-gram predicts the context words “the,” “brown,” “fox,” and “over.”
- Skip-gram is better suited for larger datasets and can capture rare words more effectively.

Why Skip-gram?

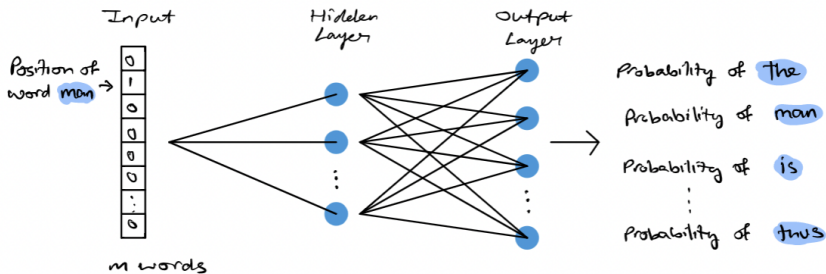
- Skip-gram is preferred for large datasets because it handles rare words more effectively than CBOW.
- It captures detailed information about the surrounding words, leading to better word representations in the vector space.
- Word2Vec embeddings from Skip-gram have been widely adopted in various NLP tasks such as machine translation, sentiment analysis, and document classification.

Skip-gram Example

Window Size	Text	Skip-grams
2	[The wide road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road shimmered in the] hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot sun].	sun, the sun, hot
3	[The wide road shimmered in] the hot sun.	wide, the wide, road wide, shimmered wide, in
	[The wide road shimmered in the hot] sun.	shimmered, the shimmered, wide shimmered, road shimmered, in shimmered, the shimmered, hot
	The wide road shimmered [in the hot sun].	sun, in sun, the sun, hot

Different window sizes and samples drawn from context words and their target

Skip-gram as a Neural Network



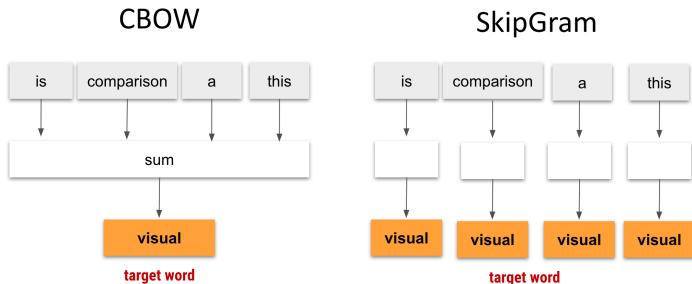
Skip-gram Model as Neural Network

Figure adapted from towardsdatascience.com/word2vec-explained-49c52b4ccb71

CBOW vs. Skip-gram

- CBOW and Skip-gram are the two primary architectures for Word2Vec.
- **CBOW** predicts a target word given its surrounding context, making it efficient and effective for smaller datasets.
- **Skip-gram**, on the other hand, predicts the surrounding words for a given target word. It is well-suited for larger datasets and can handle rare words more effectively.
- In essence, the Skip-gram model captures more detailed word relationships and is robust in large vocabularies.

CBOW vs. Skip-gram



By: Kavita Ganesan

This is a visual comparison

Difference between CBOW and Skip-gram

Skip-gram: Objective Function

- The objective of Skip-gram is to maximize the likelihood of predicting context words w_o given a center word w_c .
- The probability of a context word w_o given a center word w_c is defined as:

$$P(w_o|w_c) = \frac{\exp(v_{w_o} \cdot v_{w_c})}{\sum_x \exp(v_x \cdot v_{w_c})}$$


- v_{w_o} and v_{w_c} are the word vectors for the context and center words, respectively.

Skip-gram: Loss Function

- The goal is to minimize the negative log-likelihood over the entire training corpus:

$$J(\theta) = -\frac{1}{C} \sum_{c=1}^C \sum_{-m \leq j \leq m, j \neq 0} \log P(u_{c+j} | w_c)$$

- Here, C is the total number of words, and m is the window size.
- Skip-gram adjusts the word embeddings to maximize the probability of observing the context words around the center word.

Skip-gram: Gradient Calculation

- The detailed steps for the gradient calculation are:

$$\begin{aligned}
 \frac{\partial \log P(w_o | w_c)}{\partial v_{w_c}} &= \frac{\partial}{\partial v_{w_c}} \log \frac{e^{v_{w_o}^T v_{w_c}}}{\sum_x e^{v_x^T v_{w_c}}} \\
 &= \frac{\partial}{\partial v_{w_c}} \left(\log e^{v_{w_o}^T v_{w_c}} - \log \sum_x e^{v_x^T v_{w_c}} \right) \\
 &= v_{w_o} - \frac{1}{\sum_x e^{v_x^T v_{w_c}}} \sum_x v_x e^{v_x^T v_{w_c}} \\
 &= v_{w_o} - \sum_x P(w_x | w_c) v_x
 \end{aligned}$$

- The update rule for v_{w_c} is:

$$v_{w_c} \leftarrow v_{w_c} + \eta \left(v_{w_o} - \sum_x P(w_x | w_c) v_x \right)$$

- Here, η is the learning rate.



Skip-gram Example with Gradient Calculation

- Consider the sentence: "Cats chase mice in the dark night."
- If the center word is "chase", the model predicts context words such as "cats", "mice", "in", "the", and "dark".
- Skip-gram calculates the probability of a context word w_o given the center word w_c
- Example:
 - Word vectors: $v_{chase} = [0.1, 0.2]$, $v_{cats} = [0.3, 0.4]$, $v_{mice} = [0.2, 0.5]$.
 - For $w_o = "cats"$: Compute dot product $v_{w_o}^T v_{w_c} = (0.3)(0.1) + (0.4)(0.2) = 0.11$.
 - Softmax denominator: $\exp(0.11) + \exp((0.2)(0.1) + (0.5)(0.2)) = 1.116 + 1.148 = 2.264$.
 - Probability: $P("cats"|"chase") = \frac{1.116}{2.264} \approx 0.493$.
- Gradient for v_{chase} : $\nabla v_{w_c} = \sum_w (P(w|w_o) - y_w) v_w$, where $y_w = 1$ if $w = w_c$, else 0.

Skip-Gram Pseudocode

Algorithm 1 Skip-Gram Model

Require: Corpus D , window size w , embedding dimension d , learning rate α , number of epochs n

Initialize word embeddings W and C randomly, where W maps words to embeddings and C maps context words to embeddings

for each epoch in 1 to n **do**

for each sentence S in D **do**

for each word w_t in S **do**

 Extract context words within window size w around w_t

for each context word c of w_t **do**

 Compute dot product score $= W(w_t) \cdot C(c)$

 Compute probability $P(c|w_t)$ using softmax:

$$P(c|w_t) = \frac{\exp(\text{score})}{\sum_{c' \in \text{vocab}} \exp(W(w_t) \cdot C(c'))}$$

 Calculate loss $L = -\log P(c|w_t)$

 Update $W(w_t)$ and $C(c)$ using gradient descent with learning rate η

end for

end for

end for

return Word embeddings W

Why Do We Need Negative Sampling?

- The softmax function normalizes over all words in the vocabulary V , which can be very large (millions of words).
- This makes the calculation of $\sum_{w \in V} \exp(v_w \cdot v_{w_c})$ expensive, as it requires summing over all words in the vocabulary.
- **Solution:** Use **Negative Sampling** to only update a few randomly chosen "negative" words instead of the entire vocabulary.

Skip-gram: Negative Sampling



- **Negative sampling** involves sampling k "negative" words that do not appear in the context of the target word for each positive pair (center word and context word).
- Instead of maximizing the probability of all words in the vocabulary, we only maximize the probability of the context words and minimize the probability of the negative sampled words.
- Example: If the center word is "cat", and the context word is "cute", we sample negative words like "computer", "sky", and "table" to minimize their probability in this context.

① Introduction

② Word2Vec

Continuous Bag of Words (CBOW)

Skip-gram Model

Word Embedding Visualization

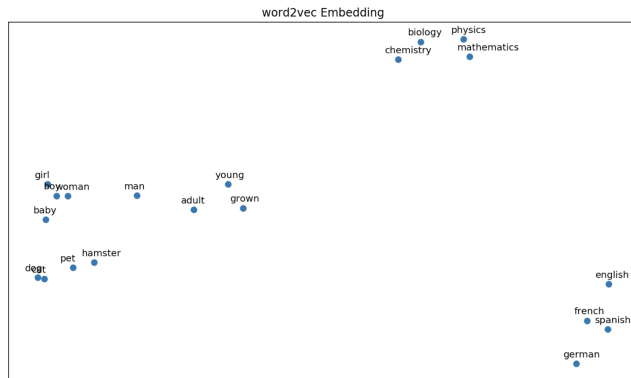
Word Analogy

③ References

Visualizing Words in 2D

- After training the model, words are mapped into a high-dimensional vector space.
- Using techniques like PCA or t-SNE, these vectors can be reduced to 2D for visualization, where similar words appear closer together.

Visualizing Words in 2D



Words represented in a 2D space after dimensionality reduction

① Introduction

② Word2Vec

Continuous Bag of Words (CBOW)

Skip-gram Model

Word Embedding Visualization

Word Analogy

③ References

Word Analogy: Vector Arithmetic in Word2Vec

- Word2Vec embeddings can solve analogy tasks by performing vector arithmetic.
- The analogy task takes the form:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

- The analogy is solved by finding the word vector closest to $\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman}$.

Word Analogy Formula

- The formal formula for solving word analogies is:

$$d = \arg \max_i \frac{(x_b - x_a + x_c)^T x_i}{\|x_b - x_a + x_c\|}$$

- Definitions:
 - x_a : Vector for the first word (e.g., "man").
 - x_b : Vector for the second word (e.g., "woman").
 - x_c : Vector for the third word (e.g., "king").
 - x_i : Candidate word vectors in the vocabulary.
- Example: To solve "man is to woman as king is to ?", the formula computes a vector closest to $x_b - x_a + x_c$ (e.g., "queen").

Word Analogy Example

- Example:

$$\text{king}[0.30, 0.70] - \text{man}[0.20, 0.20] + \text{woman}[0.60, 0.30] \approx \text{queen}[0.70, 0.80]$$

- This means the vector difference between "king" and "man" is similar to the difference between "queen" and "woman."

Contribution

- **These slides were prepared with contribution from:**
 - Omid Daliran

